

Algorithms Project

Longest Balanced Substring

Non-Recursive & Recursive Algorithms

Problem Statement

A string is balanced if it consists of exactly two different characters and both appear exactly the same number of times. A substring is a sequence of consecutive characters. Goal: find the length of the longest balanced substring in string S.

Examples:

"aabbab" → balanced (a appears 3 times, b appears 3 times)

"aabba" → NOT balanced (a appears 3 times, b appears 2 times)

"aabbcc" → NOT balanced (contains 3 distinct characters)

Task: Given a string S, find the length of the longest balanced substring.

Test Cases:

S = "cabbacc" → 4 ("abba" is the longest balanced substring)

S = "abababa" → 6 ("ababab" and "bababa" are both longest)

S = "aaaaaaa" → 0 (no balanced substring exists)

First Algorithm — Non-Recursive

1.1 Non – Recursive Pseudo Code

```
1. Algorithm GetLongestBalancedSubstring(text)
2. // text is a string
3. {
4.  n ← length(text); best ← 0;

5.  for start ← 0 to n-1 do {
6.    c1 ← NULL; c2 ← NULL; cnt1 ← 0; cnt2 ← 0;

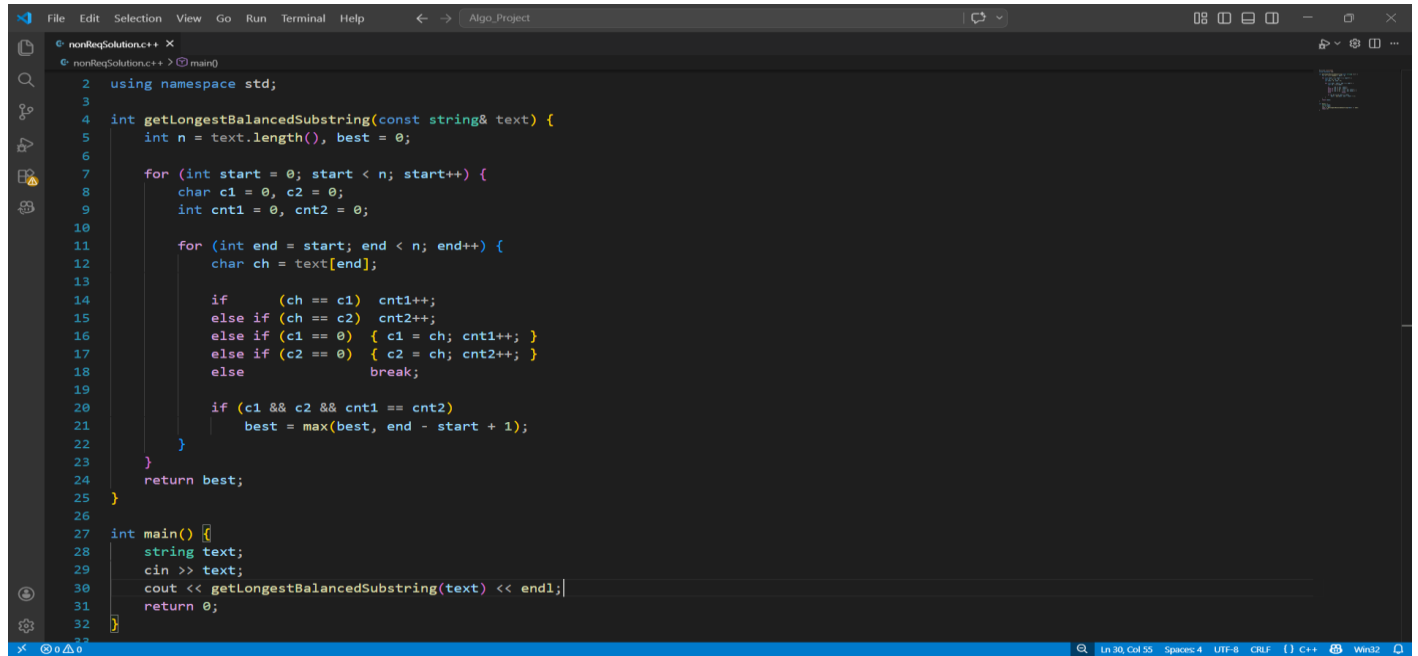
7.    for end ← start to n-1 do {
8.      ch ← text[end];

9.      if ch = c1 then cnt1 ← cnt1 + 1;
10.     else if ch = c2 then cnt2 ← cnt2 + 1;
11.     else if c1 = NULL then { c1 ← ch; cnt1 ← 1; }
12.     else if c2 = NULL then { c2 ← ch; cnt2 ← 1; }
13.     else break;

14.     if (c1 ≠ NULL AND c2 ≠ NULL AND cnt1 = cnt2) then
15.       best ← max(best, end - start + 1);
16.   }
17. }

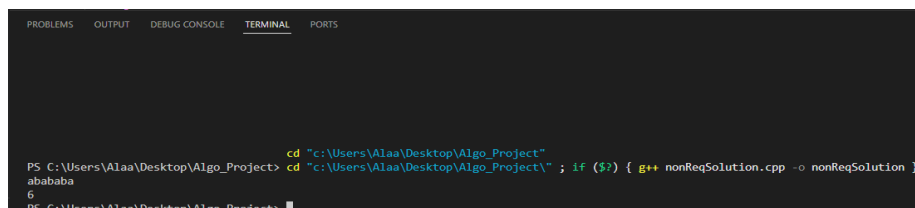
18. return best;
19. }
```

1.2 Non – Recursive Implementation (C++)

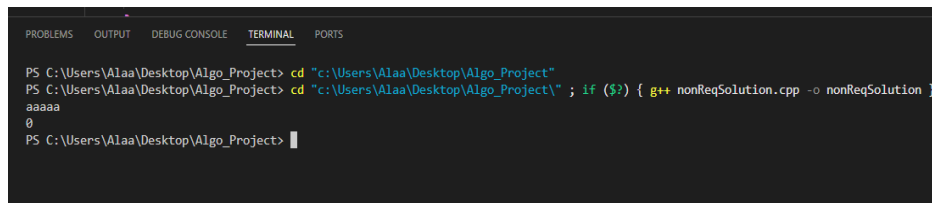


```
1  nonReqSolution.cpp X
2  using namespace std;
3
4  int getLongestBalancedSubstring(const string& text) {
5      int n = text.length(), best = 0;
6
7      for (int start = 0; start < n; start++) {
8          char c1 = 0, c2 = 0;
9          int cnt1 = 0, cnt2 = 0;
10
11         for (int end = start; end < n; end++) {
12             char ch = text[end];
13
14             if (ch == c1) cnt1++;
15             else if (ch == c2) cnt2++;
16             else if (c1 == 0) { c1 = ch; cnt1++; }
17             else if (c2 == 0) { c2 = ch; cnt2++; }
18             else break;
19
20             if (c1 && c2 && cnt1 == cnt2)
21                 best = max(best, end - start + 1);
22         }
23     }
24     return best;
25 }
26
27 int main() {
28     string text;
29     cin >> text;
30     cout << getLongestBalancedSubstring(text) << endl;
31     return 0;
32 }
```

1.3 Non – Recursive Test



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project"
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project\" ; if ($?) { g++ nonReqSolution.cpp -o nonReqSolution }
abababa
6
PS C:\Users\Alaa\Desktop\Algo_Project>
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project"
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project\" ; if ($?) { g++ nonReqSolution.cpp -o nonReqSolution }
aaaaa
0
PS C:\Users\Alaa\Desktop\Algo_Project>
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project"
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project\" ; if ($?) { g++ nonReqSolution.cpp -o nonReqSolution } ; i
aabbcc
4
PS C:\Users\Alaa\Desktop\Algo_Project>
```

1.4 Analysis & Complexity (Steps)

$$T(n) = \sum_{start=0}^{n-1} \sum_{end=start}^{n-1} 1$$

$$T(n) = \sum_{end=start}^{n-1} 1 = (n - start)$$

$$T(n) = \sum n - \sum start$$

$$T(n) = \sum_{start=0}^{n-1} n = n^2 \quad \sum_{start=0}^{n-1} start = \frac{n(n-1)}{2}$$

$$T(n) = n^2 - \frac{n(n-1)}{2} = \frac{n^2 + n}{2}$$

$$T(n) = O(n^2)$$

property	Value
Time Complexity	$O(n^2)$
Space Complexity	$O(1)$
Best Case	$O(n)$ — 3rd char appears immediately
Worst Case	$O(n^2)$ — all chars are same or two chars

Second Algorithm — Recursive

2.1 Pseudo Code

```
1. algorithm IsBalanced(s, l, r)
2. {
3.   c1 := 0; c2 := 0; cnt1 := 0; cnt2 := 0;
4.   for i := l to r do {
5.     ch := s[i];
6.     if ch = c1 then cnt1 := cnt1 + 1;
7.     else if ch = c2 then cnt2 := cnt2 + 1;
8.     else if c1 = 0 then { c1 := ch; cnt1 := 1; }
9.     else if c2 = 0 then { c2 := ch; cnt2 := 1; }
10.    else return FALSE;
11.  }
12.  return (c1 != 0 AND c2 != 0 AND cnt1 = cnt2);
13. }

14. algorithm Solve(s, start)
15. {
16.  n := length(s);
17.  if start = n then return 0;
18.  best := 0;
19.  for end := start to n - 1 do {
20.    if IsBalanced(s, start, end) = TRUE then
21.      if (end - start + 1) > best then best := end - start + 1;
22.  }
23.  return max(best, Solve(s, start + 1));
24. }

25. algorithm LongestBalanced(s)
26. {
27.  return Solve(s, 0);
28. }

29. algorithm Main
30. {
31.  read s;
32.  write LongestBalanced(s);
33. }
```

2.2 Recursive Implementation (C++)

```
File Edit Selection View Go Run Terminal Help
Algo_Project

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 bool isBalanced(const string& s, int l, int r) {
6     char c1 = 0, c2 = 0;
7     int cnt1 = 0, cnt2 = 0;
8
9     for (int i = l; i <= r; i++) {
10         char ch = s[i];
11
12         if (ch == c1) cnt1++;
13         else if (ch == c2) cnt2++;
14         else if (c1 == 0) { c1 = ch; cnt1++; }
15         else if (c2 == 0) { c2 = ch; cnt2++; }
16         else return false;
17     }
18
19     return (c1 != 0 && c2 != 0 && cnt1 == cnt2);
20 }
21
22 int solve(string s, int start) {
23     int n = s.size();
24     if (start == n) return 0;
25
26     int best = 0;
27
28     for (int end = start; end < n; end++) {
29         if (isBalanced(s, start, end))
30             best = max(best, end - start + 1);
31     }
32
33     return max(best, solve(s, start + 1));
34 }
35
36 int longestBalanced(string s) {
37     return solve(s, 0);
38 }
39
40 int main() {
41     string s;
42     cin >> s;
43
44     cout << longestBalanced(s) << endl;
45 }
```

2.3 Recursive Test

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Alaa\Desktop\Algo_Project>
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project"
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project\" ; if ($?) { g++ ReqSolution.cpp -o ReqSolution } ; if ($?) { .\ReqSolution }
abababa
6
PS C:\Users\Alaa\Desktop\Algo_Project>
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project"
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project\" ; if ($?) { g++ ReqSolution.cpp -o ReqSolution } ; if ($?) { .\ReqSolution }
cabbacc
4
PS C:\Users\Alaa\Desktop\Algo_Project>
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cd "c:\Users\Alaa\Desktop\Algo_Project"
2
PS C:\Users\Alaa\Desktop\Algo_Project> cd "c:\Users\Alaa\Desktop\Algo_Project\" ; if ($?) { g++ ReqSolution.cpp -o ReqSolution } ; if ($?) { .\ReqSolution }
aaaaaaa
0
PS C:\Users\Alaa\Desktop\Algo_Project>
```

2.4 Analysis & Complexity (Steps)

Iteration method

(I) Iteration method

$$T(n) = T(n-1) + n^2$$

$$T(n-1) = T(n-2) + (n-1)^2$$

$$T(n-2) = T(n-3) + (n-2)^2$$

$$T(n-3) = T(n-4) + (n-3)^2$$

$$T(n) = T(n-4) + (n-3)^2 + (n-2)^2 + (n-1)^2 + n^2$$

$$T(n) = T(n-k) + (n-(k-1))^2 + (n-(k-2))^2 + (n-(k-3))^2 + \dots + (n-1)^2 + n^2$$

$$n = k$$

$$T(n-n) + (n-(n+1))^2 + (n-(n-2))^2 + \dots + (n-1)^2 + n^2$$

$$T(n) = T(0) + 1^2 + 2^2 + \dots + (n-1)^2 + n^2$$

$$T(n) = 1 + \sum_{i=1}^n i^2$$

$$T(n) = 1 + \frac{n(n+1)(2n+1)}{6}$$

$$T(n) = 1 + \frac{(n^2+n)(2n+1)}{6} = 1 + \frac{2n^3 + n^2 + 2n^2 + n}{6}$$

$$= 1 + \frac{2n^3 + n^2 + 2n^2 + n}{6} \rightarrow \frac{1 + 2n^3 + 3n^2 + n}{6}$$

$$\approx O(n^3) \text{ . The complexity of Recursion code}$$

Master method & iteration tree

التاريخ : الموضوع :

(2) master method decreasing "master"

$$T(n) = T(n-1) + n^2$$

$$a=1 \quad b=1 \quad f(n) = n^2 \quad \therefore a=1$$

$$F(n) = O(n^k) \quad k \geq 0 \quad k=2$$

$$T(n) = O(n^{k+1}) \rightarrow O(n^{2+1}) = O(n^3)$$

The time complexity of recursion code $O(n^3)$

(3) Tree method

$$T(n) = T(n-1) + n^2$$

$$\Rightarrow 0 + 1^2 + 2^2 + 3^2 + \dots + (n-1)^2 + n^2$$

$$= \sum_{i=1}^n i^2 = \frac{n(n+1)(n+2)}{6}$$

$$= \frac{2n^3 + 3n^2 + n}{6}$$

$$\approx O(n^3)$$

$T(n) = \begin{cases} 1 & n=0 \\ T(n-1) + n^2 & n>0 \end{cases}$

another solution from The algorithm

Property	Value
Recurrence	$T(n) = T(n-1) + O(n^2)$
Time Complexity	$O(n^3)$
Space Complexity	$O(n)$ — call stack depth
Best Case	$O(n^2)$ — early breaks
Worst Case	$O(n^3)$ — no early breaks

Comparison — Iterative vs Recursive

Criterion	Iterative (Non-Recursive)	Recursive
Time Complexity	$O(n^2)$	$O(n^3)$
Space Complexity	$O(1)$	$O(n)$ — call stack
Stack Overflow	No risk	Risk on large inputs
Readability	Clear & direct	Harder to trace
Redundant Work	None	Rescans every [start..end]
Performance	Faster	Slower
Best for	Production use	Conceptual understanding

Key Insight

Both algorithms solve the same problem correctly. The fundamental difference lies in how they re-use computed information:

Iterative: counts c1 and c2 incrementally as 'end' grows, never rescanning already-visited characters. This gives $O(n^2)$.

Recursive: at every call `RecursiveSolve(text, start, end)` it rescans the entire [start..end] range from scratch. This extra $O(n)$ scan per call elevates the total to $O(n^3)$.

Conclusion: the Iterative algorithm is strictly superior in both time and space for this problem.